System Architecture

System Architecture

1. Overview

The EdgeAI Predictive Maintenance system follows a modular, layered architecture. Each layer has a single, well-defined responsibility, and data flows from the physical machine through embedded firmware, signal processing, TinyML inference, communication, and visualization. A rendered version of this document is also available as `System_Architecture.pdf`.

2. Complete System Workflow

```
Industrial Machine
      |
      ▼
Sensors (Vibration + Acoustic)
      |
      ▼
ESP32 Data Acquisition
      |
      ▼
Signal Preprocessing
      |
      ▼
Feature Extraction
      |
      ▼
TinyML Inference
      |
      ▼
Health Classification
      |
      ├─► OLED Display
      ├─► LEDs
      ├─► Buzzer
      |
      ▼
MQTT Publisher → WiFi → MQTT Broker → Dashboard → Real-Time
Monitoring
```

3. Layered Architecture

```
+--------------------------------------+
|        Dashboard Application         |
+--------------------------------------+
|      Communication (MQTT/WiFi)       |
+--------------------------------------+
|      TinyML Inference Engine         |
+--------------------------------------+
|      Signal Processing Layer         |
+--------------------------------------+
|      Sensor Abstraction Layer        |
+--------------------------------------+
|      Hardware Drivers Layer          |
+--------------------------------------+
|      Industrial Machine              |
+--------------------------------------+
```

Each layer communicates only with adjacent layers, which significantly improves maintainability and testability.

4. Hardware Architecture

Primary Controller: ESP32 — reads sensors, executes DSP and TinyML inference, drives OLED/LEDs/buzzer, publishes MQTT. **Vibration Sensor / Accelerometer** — measures mechanical vibration to detect bearing wear, rotor imbalance, loose mounting, shaft misalignment. **INMP441 MEMS Microphone (I²S)** — captures acoustic emissions such as bearing noise, gear noise, mechanical impacts, and high-frequency abnormalities. **OLED Display** — local feedback: machine state, prediction, signal level, connection status. **LEDs** — Green (Healthy) / Yellow (Warning) / Red (Fault). **Buzzer** — audible alert, activated only during critical faults. **Wi-Fi** — transfers health status to the dashboard.

5. Firmware Architecture

```
main()
 │
 ├── System Initialization
 ├── Sensor Manager
 ├── Signal Processing
 ├── TinyML Manager
 ├── Display Manager
 ├── MQTT Manager
 ├── Alarm Manager
 └── Diagnostics
```

Each module owns exactly one responsibility (Single Responsibility Principle). See `firmware/README.md` for the full breakdown.

6. Firmware Execution Sequence

```
Power ON → Initialize Hardware → Initialize Sensors → Initialize
WiFi →
Initialize MQTT → Collect Sensor Data → Filter Signals → Extract
Features →
Run TinyML → Determine Machine Health → Update OLED → Update LEDs →
Trigger Alarm if Required → Publish MQTT → Repeat
```

7. Sensor Abstraction Layer

```
Application → Sensor Manager → Drivers → ESP32 HAL → Physical
Sensors
```

Isolating hardware access behind a sensor manager gives cleaner code, easier debugging, easier sensor replacement, and higher portability.

8. Signal Processing Layer

```
Raw Signal → DC Removal → Noise Filtering → Normalization →
Windowing → FFT → Feature Extraction → TinyML
```

Time-domain features: RMS, Peak, Mean, Variance, Crest Factor (Kurtosis/Skewness planned). Frequency-domain features: Dominant Frequency, Frequency Energy, Spectral Peaks, Frequency Band Energy.

9. TinyML Architecture

```
Features → TensorFlow Lite Model → Inference → Prediction →
Confidence Score → Machine State
```

Outputs: Healthy / Warning / Fault.

10. Communication Architecture

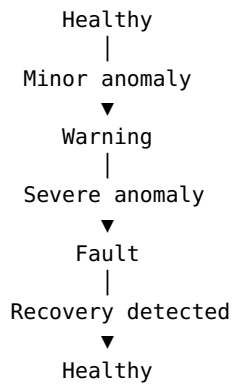
```
ESP32 → MQTT Publish → Broker → Dashboard → Visualization
```

Publish-subscribe model gives low bandwidth usage, scalability, multi-machine support, and easy dashboard integration.

11. Dashboard Architecture

```
MQTT → Backend → Data Parser → Frontend → Charts → User
```

12. Machine State Diagram



This finite-state approach prevents unstable rapid switching between states.

13. Data Flow Diagram

```
Machine → Sensors → ESP32 ADC/I2S → Signal Processing → Feature
Extraction →
TinyML → Classification → OLED → MQTT → Dashboard
```

14. Error Handling Strategy

The firmware continuously checks for: sensor not detected, WiFi disconnected, MQTT unavailable, memory allocation failure, inference failure, invalid sensor values. Each error is logged and handled without crashing the system whenever possible.

15. Scalability

Future additions: temperature sensing, current sensing, voltage monitoring, multi-axis vibration sensors, multi-machine support, OTA updates, edge anomaly detection, RUL estimation, cloud sync, mobile app, digital twin integration.

16. Design Principles

- **Modularity** — each subsystem has a single responsibility
- **Scalability** — new sensors/features added with minimal changes
- **Maintainability** — clear separation of hardware, firmware, AI, and visualization layers
- **Portability** — hardware abstraction allows adaptation to different microcontrollers
- **Reliability** — error handling and modular testing improve robustness
- **Edge-first Processing** — critical decisions made locally to reduce latency and network dependence